Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: RICCARDO POLO Discussed with:

Report for Project 2

Due date: 23 March 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Computing π with OpenMP [20 points]

1.1. Exercise 1

In this exercise we are given a code that computes an approximation of π using numerical integration. The code uses a simple loop to compute the sum of terms that approximate the value of π. The goal of this exercise is to parallelize the code using `OpenMP` and to optimize it for better performance.

In the first part of the exercise it was asked to first parallelize the given code using a naive version that protects every loop update with `critical`. Here is the code snippet of the naive version that we implemented:

```
#pragma omp parallel for
for (uint64_t i = 0; i < N; ++i) {
  double x = (i + 0.5) * h;
  double term = 4.0 / (1.0 + x*x);

  #pragma omp critical
  pi += term;
}
pi *= h;
```

The naive version is very slow. This is because every update to the shared variable `pi` is protected by a critical section, which causes a lot of contention and serialization among the threads. Each thread has to wait for the others to finish updating `pi`, which leads to a significant slowdown in performance.

In the second part it was asked to create an optimized version that accumulates locally and enters critical only once per thread. Here is the code snippet of the optimized version that we implemented:

```
#pragma omp parallel
{
  double sum = 0.0;
  #pragma omp for
  for (uint64_t i = 0; i < N; ++i) {
    double x = (i + 0.5) * h;
    sum += 4.0 / (1.0 + x*x);
  }
  #pragma omp critical
  pi += sum; // once per thread
}
pi *= h;
```

In this version we have a local variable `sum` that is private to each thread, and we only enter the critical section once per thread to update the shared variable `pi`. This reduces the contention on the critical section and allows for better performance.

We are then asked to implement a version that uses the reduction clause. Here is the code snippet of the version that uses the reduction clause:

```
#pragma omp parallel for reduction(+:sum)
  for (uint64_t i = 0; i < N; ++i) {
    double x = (i + 0.5)*h;
    sum += 4.0 / (1.0 + x*x);
  }
  pi = sum*h;
```

In this version we use the `reduction` clause to automatically handle the accumulation of the variable `pi` across threads. The reduction clause takes care of creating a private copy of the variable for each thread and then combining the results at the end of the parallel region. This allows for a more concise and efficient implementation.
In table 1 we summarize the results of the different versions of the code.

Version	π	num_threads	time [secs]
Serial	3.141592653589971	1	1.88825762
Naive	3.141592653590757	64	640.706
Optimized	3.141592653589771	64	0.19858712
Reduction	3.141592653589771	64	0.11437800

Table 1: Results of the different versions of the code for computing π

1.2. Exercise 2

In this part of the exercise we perform a **strong** and **weak scaling study** of the three implementations of the π computation.

1.2.1. Strong scaling study

In a **strong scaling study**, the total problem size is kept constant while the number of threads p is increased. In our case, the number of integration steps is fixed to $N = 10^9$.
For each number of threads we measure the execution time T_p. From these measurements we compute the **speedup**, defined as in equation 1

$$S_p = \frac{T_1}{T_p} \tag{1}$$

where T_1 is the execution time with one thread.
The ideal speedup is given by equation 2

$$S_{ideal} = \frac{n\,p}{n+p-1} \approx n, \tag{2}$$

where $n = N$ is the number of loop iterations, and p is the number of threads. In practice, we expect to see a speedup that is less than ideal due to overheads such as thread management and synchronization. The results of the strong scaling experiment are reported in Table 2.

Table 2: Strong scaling results for the different implementations ($N = 10^9$).

Threads	Naive Critical		Optimized Critical		Reduction	
	Time [s]	Speedup	Time [s]	Speedup	Time [s]	Speedup
1	6.4987	1.00	1.4189	1.00	1.3933	1.00
2	61.7839	0.11	0.7696	1.84	0.7573	1.84
4	105.7615	0.06	0.4260	3.33	0.4268	3.26
8	131.8628	0.05	0.2179	6.51	0.2179	6.39
16	391.7679	0.02	0.1575	9.01	0.1574	8.86
32	427.1595	0.02	0.0789	17.98	0.0789	17.66
64	600.1213	0.01	0.0449	31.60	0.0448	31.06

From the results we observe a very different behavior among the three implementations.

The **naive implementation with a critical section** performs extremely poorly. As the number of threads increases, the execution time increases dramatically instead of decreasing. This is due to the fact that every iteration of the loop enters a `critical` section to update the shared variable. As a consequence, the threads serialize their execution and spend most of the time waiting to acquire the lock. This creates severe contention and eliminates the benefits of parallelization.

The **optimized critical version** performs significantly better. In this implementation each thread accumulates a local partial sum and only enters the `critical` section once at the end of its work. This drastically reduces the synchronization overhead. As a result, the speedup increases almost linearly up to several threads.

The **reduction implementation** shows a very similar behavior to the optimized version. This is expected since OpenMP reductions internally implement an efficient private accumulation followed by a final combination step. The reduction version reaches a speedup of about 31 with 64 threads, showing good parallel scalability.

1.2.2. Weak scaling study

In a **weak scaling study**, the amount of work per thread is kept constant while the total problem size grows proportionally with the number of threads. If p is the number of threads, the total problem size becomes $N = N_0 \times p$ where N_0 is the number of iterations assigned to each thread. In our experiments we use $N_0 = 5 \times 10^7$.
The main metric used to evaluate weak scaling is the **parallel efficiency**, defined as in eq. 3.

$$E_p = \frac{T_1}{T_p} \tag{3}$$

where T_p is the execution time with p threads and problem size $N = N_0 p$. Ideally, the execution time should remain constant as the number of threads increases, resulting in $E_p \approx 1$. The results of the weak scaling experiment are shown in Table 3.

Table 3: Weak scaling results for the different implementations (work per thread = 5×10^7).

Threads	N	Naive Critical		Optimized Critical		Reduction	
		Time [s]	Efficiency	Time [s]	Efficiency	Time [s]	Efficiency
1	5×10^7	0.4024	1.00	0.1260	1.00	0.0758	1.00
2	1×10^8	5.4202	0.07	0.1203	1.05	0.0956	0.79
4	2×10^8	23.1030	0.02	0.1166	1.08	0.0924	0.82
8	4×10^8	88.4441	0.00	0.0943	1.34	0.0909	0.83
16	8×10^8	225.6566	0.00	0.1261	1.00	0.1262	0.60
32	1.6×10^9	616.5533	0.00	0.1265	1.00	0.1275	0.59
64	3.2×10^9	2015.6843	0.00	0.1325	0.95	0.1265	0.60

The **naive implementation** scales extremely poorly. The execution time increases dramatically with the number of threads due to the heavy contention on the `critical` section. Since every iteration requires synchronization, the amount of waiting grows rapidly as more threads are added.

The **optimized critical version** shows very good weak scaling behavior. The execution time remains approximately constant as the number of threads increases, with efficiency values close to one. This indicates that the implementation can handle larger problem sizes effectively while maintaining good parallel performance.

The **reduction implementation** also exhibits good weak scaling, although the efficiency slightly decreases for larger thread counts. This behavior is expected due to additional overheads such as thread management and reduction synchronization.

The data reported are obtained by taking the average of 5 different runs. In this way we can mitigate the effects of variability in execution time and obtain more reliable measurements. By looking at the outputs in the `.out` files one can also see the minimum and maximum times recorded across the runs, which gives an indication of the variability in performance.

1.3. Use of LLMs

Chat gpt was used to create the tables in Latex from the data collected from the experiments and to confirm the correcteness of the equations used for speedrun and efficiency calculations.
It was also used to properly modify the .c files to modify correctly the original codes in order to perform a strong scaling and a weak scaling study.

2. The Mandelbrot set using OpenMP [20 points]

2.1. Exercise 1 and 2

Here we are asked to complete the serial version of the Mandelbrot set code in the missing code part and to count the total number of iterations in order to correctly compute the benchmark statistics.
In the following code snippet we show the completed version of the serial code for computing the Mandelbrot set:

```
double time_start = walltime();
  // do the calculation
  cy = MIN_Y;
  for (j = 0; j < IMAGE_HEIGHT; j++) {
    cx = MIN_X;
    for (i = 0; i < IMAGE_WIDTH; i++) {
      x = cx;
      y = cy;
      x2 = x * x;
      y2 = y * y;
      int n = 0;

      while (x2 + y2 <= 4.0 \&\& n < MAX\_ITERS) {
        y = 2.0 * x * y + cy;
        x = x2 - y2 + cx;
        x2 = x * x;
        y2 = y * y;
        n++;
      }
       nTotalIterationsCount += n;
      // plot the number of iterations at point (i, j)
      int c = ((long)n * 255) / MAX_ITERS;
      ppm_plot(&pPpm, i, j, c, c, c);
      cx += fDeltaX;
  }
    cy += fDeltaY;
  }
  double time_end = walltime();
```

In the while loop, we are iterating through the orbit of the point c under the function $f(z) = z^2 + c$. The variables x and y represent the real and imaginary parts of the complex number z, respectively. The function $f(z)$ can be expressed in terms of its real and imaginary components as follows:

$$\begin{aligned} f(z) &= z^2 + c \\ &= (x + yi)^2 + (c_x + c_y i) \\ &= (x^2 - y^2 + c_x) + (2xy + c_y)i \end{aligned}$$

Since the function $f(z)$ is defined recursively, and each iteration of the function applies the same transformation to the complex number z. Therefore, when we compute $f^2(z)$, we are applying the function f to the result of $f(z)$, which means that we are applying the same equations for the real and imaginary parts at each iteration.
From this, we can see that the real part of $f(z)$ is given by $x^2 - y^2 + c_x$, and the imaginary part is given by $2xy + c_y$. The loop continues until the magnitude of z (which is $\sqrt{x^2 + y^2}$) exceeds 2, indicating that the orbit is unbounded, or until we reach the maximum number of iterations, which indicates that the point c is likely to belong to the Mandelbrot set.
Finally, we are asked to count the total number of iterations in order to correctly compute the benchmark statistics. This is done by introducing a variable `nTotalIterationsCount` that accumulates the number of iterations for each point in the complex plane. This variable is incremented by n, which represents the number of iterations for the current point (i, j).

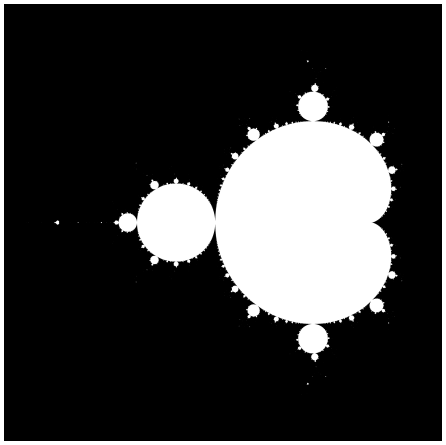


Figure 1: Serial Mandelbrot Set

2.2. Exercise 3

In the third part of the exercise we are asked to parallelize the Mandelbrot set code using `OpenMP`.. Here is the code snippet of the parallelized version of the Mandelbrot set code using `OpenMP`:

```
double time_start = walltime();
#pragma omp parallel for reduction(+:nTotalIterationsCount) schedule(dynamic)
for (long j = 0; j < IMAGE_HEIGHT; j++) {
    double cy = MIN_Y + j * fDeltaY;

  for (long i = 0; i < IMAGE_WIDTH; i++) {
      double cx = MIN_X + i * fDeltaX;

    double x = cx;
    double y = cy;
    double x2 = x * x;
    double y2 = y * y;

    int n = 0;
    while (x2 + y2 <= 4.0 && n < MAX_ITERS) {
      y = 2.0 * x * y + cy;
      x = x2 - y2 + cx;
      x2 = x * x;
      y2 = y * y;
      n++;
    }

    nTotalIterationsCount += n;

    int c = ((long)n * 255) / MAX_ITERS;
    ppm_plot(&pPpm, i, j, c, c, c);
  }
}
double time_end = walltime();
```

To parallelize we used the `parallel for` directive to distribute the iterations of the outer loop (over j) among the threads. We also used a reduction clause to safely accumulate the total number of iterations across all threads. The schedule is set to dynamic to allow for better load balancing, as some points in the complex plane may require more iterations than others.
We have also chanced the way with initialize the variables `cx` and `cy` to make it work in the parallel version. We compute their values inside the loops based on the current indices i and j. This allows each thread to compute the correct values of `cx` and `cy` for its assigned iterations without any race conditions.

The results (see table 4) show a significant performance improvement achieved by parallelizing the code

Metric	Serial Version	Parallel Version
Total time [seconds]	330.726	6.70717
Total number of iterations	113610974266	113638786651
Average time per pixel [seconds]	1.97128e-05	3.99779e-07
Average time per iteration [seconds]	2.91104e-09	5.90219e-11
Iterations per second	3.4352e+08	1.69429e+10
MFlop/s	2748.16	135543

Table 4: Comparison of the serial and parallel versions of the Mandelbrot set code

using `OpenMP`. The total time taken to compute the Mandelbrot set is reduced from 330.726 seconds in the serial version to 6.70717 seconds in the parallel version, which is a substantial decrease in execution time. The total number of iterations is slightly higher in the parallel version, which could be due to differences in the order of execution and how the iterations are distributed among the threads. The average time per pixel and average time per iteration are also significantly reduced in the parallel version, indicating that the computation is much faster when utilizing multiple threads. The iterations per second and MFlop/s metrics show a dramatic increase in performance, with the parallel version achieving 1.69429e+10 iterations per second and 135543 MFlop/s compared to 3.4352e+08 iterations per second and 2748.16 MFlop/s in the serial version.

2.2.1. Strong scaling analysis

To analyze the strong scaling of the parallelized Mandelbrot set code, we can measure the execution time for different numbers of threads while keeping the problem size fixed. This allows us to see how well the code scales with increasing thread count.
We also calculate the speedup as the ratio of the execution time of the serial version to the execution time of the parallel version for each thread count. In figure 2 we show the plots of the strong scalying analysis. As we can see the execution times halves every time we double the number of threads, which indicates that the code scales well with increasing thread count. In figure 3 instead we plot the theoretical speedup with the one computed from the simulations. The ideal speedup is computed via 2. The actual speedup is computed as shown in equation 1.

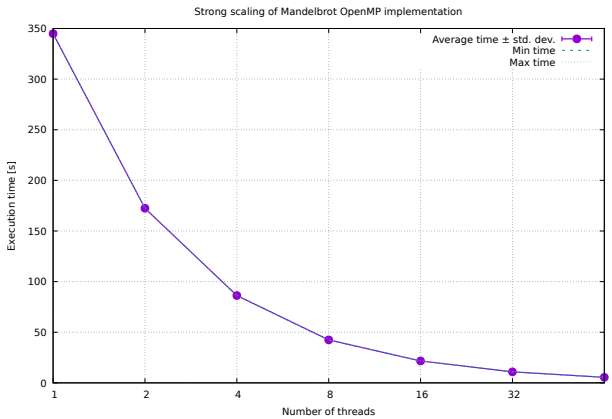


Figure 2: Strong scaling analysis of the parallelized Mandelbrot set code

Figure 2 shows the execution time of the Mandelbrot OpenMP implementation as a function of the number of threads. As expected in a strong scaling study, the problem size is fixed while the number of threads increases. The execution time always halves when moving from 1 to 64 threads, demonstrating that the parallelization is effective in exploiting multiple cores. However, beyond 16 threads the improvement becomes less pronounced and the curve starts to flatten.

The error bars representing the standard deviation are relatively small across all thread counts, indicating that the measurements are stable and reproducible. Indeed by looking at the .data file one can see that the minimum and maximum execution times remain close to the average values.
Figure 3 reports the corresponding speedup with respect to the single-thread execution. The speedup stays consistently close to the the ideal speedup curve as the number of threads increases.

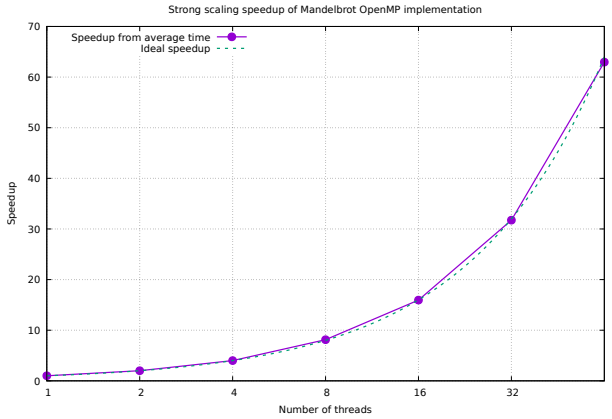


Figure 3: Speedup of the parallelized Mandelbrot set code compared to the ideal speedup

Overall, the results show that the OpenMP parallelization of the Mandelbrot computation scales well. As we did for the first exercise, the data shown in figures 2 and 3 are obtained by taking the average of 5 different runs.

2.3. Use of LLMs

Chat GPT was used to create the gnuplot files and the bash file to generate the strong scaling plots.

3. Bug hunt [10 points]

3.1. File 1: omp_bug1.c

The program attempts to parallelize the loop that computes $c[i] = a[i] + b[i]$ using the OpenMP `parallel for` construct.
The first problem is related to the use of the OpenMP directive `#pragma omp parallel for`. This combined directive creates a parallel region and distributes the iterations of the following loop among the threads. For this reason, the directive must be followed *immediately* by a `for` loop. In the program, however, the directive is followed by a compound block:

```
#pragma omp parallel for shared(a, b, c, chunk) private(i, tid) \
          schedule(static, chunk)
{
  tid = omp_get_thread_num();
  for (i = 0; i < N; i++)
```

This structure is invalid because the `parallel for` directive must be attached directly to the loop it parallelizes.

The second issue concerns the variable `tid`. The programmer wants to print which thread executes each iteration, but the statement `tid = omp_get_thread_num()` is placed outside the loop that the directive is supposed to parallelize. Because the directive requires the loop to appear immediately after it, this statement breaks the required structure of the construct.

To fix the problem, the loop must appear directly after the OpenMP directive. The thread identifier should instead be retrieved inside the loop body (or inside a parallel region if the `parallel` and `for` constructs are separated). In this way, each thread can correctly obtain its ID and print the information for the iterations it executes.

3.2. File 2: omp_bug2.c

The program contains errors related to the incorrect handling of `shared` and `private` variables inside the OpenMP parallel region.

The variables `tid`, `i`, and `total` are declared outside the `#pragma omp parallel` region and therefore are *shared* by default among all threads. This leads to race conditions and incorrect program behaviour.

First, the variable `tid` is shared, although each thread assigns its own thread ID through `tid = omp_get_thread_num()`. Since all threads write to the same memory location, the value of `tid` can be overwritten by other threads. As a result, the value stored in `tid` is not guaranteed to correspond to the correct thread.

A more serious problem concerns the variable `total`. Inside the parallel region the code initializes `total = 0.0;` and then performs a parallel loop

```
#pragma omp for schedule(dynamic, 10)
for (i = 0; i < 1000000; i++)
   total = total + i * 1.0;
```

Because `total` is shared, all threads update the same variable concurrently. This produces a race condition and the resulting sum is incorrect and non-deterministic. Moreover, each thread resets the shared variable to zero before the loop, which introduces additional data races.
The variable `tid` should be declared *private* so that each thread keeps its own thread identifier. The summation should be implemented using an OpenMP `reduction` on `total`, which safely combines the partial sums computed by different threads.

3.3. File 3: omp_bug3.c

The main problem in this program is the incorrect use of the `barrier` directive inside the function `print_results`. In OpenMP, a barrier must be encountered by *all* threads of the team, or by none of them. Here, however, `print_results` is called only by the threads that actually execute one of the two `sections`. Since there are only two sections, at most two threads will enter the function, while the other threads in the parallel region will skip it entirely. As a consequence, only a subset of threads reaches the barrier placed at the end of `print_results`, which leads to an illegal synchronization pattern and can cause the program to hang at run time.

A second issue is that the intent of having "clean output" is only partially achieved. The `critical` region correctly prevents the printed array contents from being interleaved, so the output of one thread is kept together. However, the statement

```
printf("Thread %d done and synchronized.\n", tid);
```

is placed *after* the barrier and outside the critical region, so even if the barrier were legal, these messages could still appear in overlapping or non-deterministic order with respect to other threads' output.

There is also a conceptual inconsistency in combining `sections nowait` with the later synchronization logic. The `nowait` clause removes the implicit barrier at the end of the `sections` construct, but then the code tries to enforce synchronization manually inside `print_results` using a barrier, which is not valid for the reason explained above. If synchronization after the sections is desired, it should be done at a point that is reached by every thread in the parallel region.

To fix the code, the barrier inside `print_results` should be removed. Any barrier should instead be placed in the main parallel region, at a location where all threads are guaranteed to encounter it. Moreover, if the programmer wants all output messages to appear cleanly and without overlap, every printing operation that must remain grouped should be enclosed in the same `critical` region, or alternatively all printing should be reorganized so that only one thread prints at a time in a controlled way. Finally, if synchronization at the end of the `sections` construct is required, the simplest correction is to remove the `nowait` clause and rely on the implicit barrier provided by `sections`.

In summary, the critical bug is the illegal barrier inside `print_results`, because not all threads execute that function. This may cause a deadlock. The proper fix is to move synchronization points outside the function and ensure that every barrier is reached by all threads in the team.

3.4. File 4: `omp_bug4.c`

The problem is caused by *stack overflow* in the OpenMP region. The key issue is that the array `real(8) :: e_in(nmax)` is declared inside the main program and the parallel region uses `!$omp parallel default(firstprivate)`. Because of `firstprivate`, each OpenMP thread receives its own private copy of `e_in`. Since $n_{\max} = 202000$, this array alone occupies about $202000 \times 8 \approx 1.6$ MB per thread. These private copies are typically placed on the thread stack. If the available stack size is too small, entering the parallel region causes a segmentation fault.

There is a second large object involved: `double complex :: eTDSE(ntmax)` inside the module, declared as `!$omp threadprivate(eTDSE)`.
This means that each thread also has its own copy of `eTDSE`. A `double complex` element usually requires 16 bytes, so this array takes about $202000 \times 16 \approx 3.2$ MB per thread. Therefore, with OpenMP enabled, the memory required per thread becomes quite large. The crash appears only when both arrays are effectively used in the parallel region, which is why removing one of the `print` statements may make the program run.

The code is not wrong from a syntax point of view, but it is wrong from a memory-management perspective. The main mistake is using `default(firstprivate)` in a region where a large local array such as `e_in` exists. This forces unnecessary duplication of a large array on each thread stack. In addition, `threadprivate(eTDSE)` also creates one copy per thread, which further increases memory pressure.
So the real bug is the combination of large thread-local data and insufficient thread stack size.

A first possible fix is to avoid making `e_in` firstprivate if this is not needed. Since the array is only read in the loop, it should be shared instead of private. In practice, one should replace the overly broad `default(firstprivate)` with a more careful data-sharing specification, for example making `e_in` shared.

Another possible improvement is to avoid storing such large arrays on the stack.

3.5. File 5: `omp_bug5.c`

The main bug in this program is a **deadlock caused by inconsistent lock ordering**. The two `sections` acquire the two locks in opposite order. In the first section the thread acquires `locka` and then `lockb`, while in the second section the order is reversed: the thread first acquires `lockb` and then `locka`.

This can lead to a deadlock situation. For example, Thread 0 may acquire `locka` while Thread 1 acquires `lockb`. Afterwards, Thread 0 waits for `lockb`, which is already held by Thread 1, and Thread 1 waits for `locka`, which is held by Thread 0. Since each thread is waiting for a lock owned by the other one, neither thread can proceed and the program blocks indefinitely.

There is also a second issue related to the loop variable `i`. The variable `i` is declared outside the parallel region and is therefore **shared** by default. Since both sections execute concurrently and use the same loop variable, this creates a **data race**. Each thread should instead operate on its own private copy of the loop index.

To fix the program, the locking order must be made consistent. All threads should acquire multiple locks in the same global order (for instance always acquiring `locka` before `lockb`). This removes the circular wait condition and therefore prevents the deadlock. Another possible improvement is to redesign the algorithm so that nested locking is not required; for example, the arrays could be initialized first, followed by a synchronization point, and only afterwards the addition operations could be performed. Finally, the loop variable `i` should be made **private** inside the parallel region or declared locally inside each section so that each thread uses its own independent copy.

4. Parallel histogram calculation using OpenMP [15 points]

We place the code snippet of the parallelized version of the histogram calculation using OpenMP in the following listing. The code is parallelized by using a private histogram for each thread and then combining the results at the end of the parallel region.

```
#define VEC_SIZE 1000000000
#define BINS 16
...
  time_start = walltime();

  int nthreads = omp_get_max_threads();
  std::vector<std::vector<long>> local_hist(nthreads, std::vector<long>(BINS, 0));

  #pragma omp parallel
  {
    int tid = omp_get_thread_num();

    #pragma omp for
    for (long i = 0; i < VEC_SIZE; ++i) {
      local_hist[tid][vec[i]]++;
    }
  }

  // Combine the histograms
  for (int t = 0; t < nthreads; ++t) {
    for (int b = 0; b < BINS; ++b) {
      dist[b] += local_hist[t][b];
    }
  }

  time_end = walltime();
```

In this code, we first generate a large vector of random integers that fall within the range of the histogram bins. We then initialize a histogram array to count the occurrences of each bin. To parallelize the histogram computation, we create a local histogram for each thread using a 2D vector `local_hist`, where each row corresponds to a thread and each column corresponds to a bin. We use an OpenMP parallel region to allow multiple threads to execute concurrently. Inside the parallel region, we retrieve the thread ID using `omp_get_thread_num()` and then use a parallel for loop to iterate over the input vector. Each thread updates its own local histogram based on the values in the input vector. After the parallel region, we combine the local histograms into the final histogram `dist` by summing the counts from each thread for each bin. Finally, we record the end time to measure the execution time of the histogram computation. This approach avoids contention on shared histogram bins by allowing each thread to maintain its own local counts, which are then combined at the end, resulting in improved performance and scalability.

4.1. Strong scaling analysis

To analyze the strong scaling of the parallelized histogram calculation, we can measure the execution time for different numbers of threads while keeping the problem size fixed. This allows us to see how well the code scales with increasing thread count. We can also calculate the speedup as the ratio of the execution time of the serial version to the execution time of the parallel version for each thread count. The ideal speedup is as usual given by eq. 2 and the actual speedup is given by eq. 1. In figure 4 we show the execution time for different numbers of threads and in figure 5 we show the speedup compared to the ideal speedup.
The data shown in the plots are obtained by taking the average of 5 different runs. This helps to mitigate the effects of variability in execution time and provides more reliable measurements. By looking at the outputs in the `timing_hist_omp_strong.data` file one can also see the minimum and maximum times recorded across the runs, which gives an indication of the variability in performance.

From figure 4 we observe that the execution time decreases rapidly when increasing the number of threads from 1 up to about 8 threads, indicating that the parallelization successfully exploits the available cores in

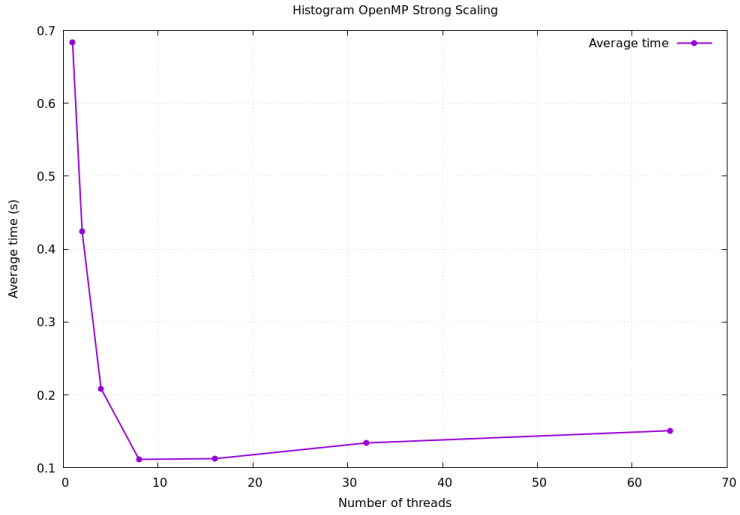


Figure 4: Strong scaling analysis of the parallelized histogram calculation

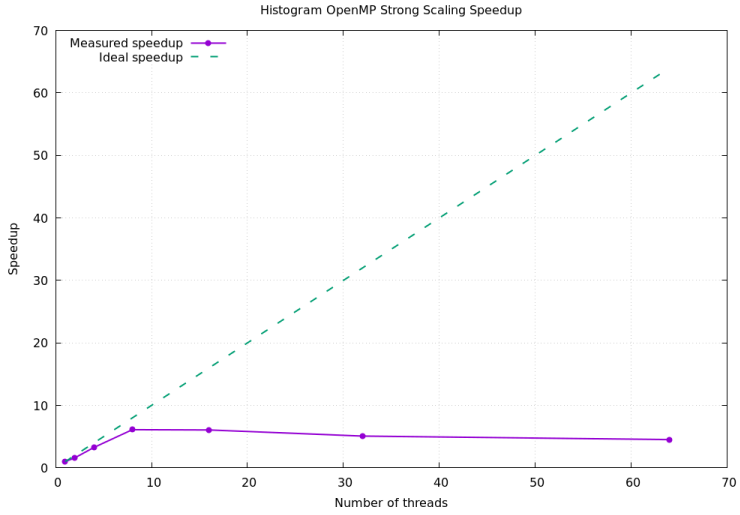


Figure 5: Speedup of the parallelized histogram calculation compared to the ideal speedup

this regime. The minimum execution time is reached around 8-16 threads. However, when increasing the number of threads beyond this point, the execution time no longer decreases and even starts to increase slightly.

This behavior is reflected in the speedup plot shown in figure 5. The speedup initially grows with the number of threads but quickly deviates from the ideal linear scaling. The maximum measured speedup is around 6×, after which the speedup saturates and slightly decreases for larger thread counts.

The loss of scalability at higher thread counts is likely due to memory-system effects. The algorithm performs a very small amount of arithmetic per input element and is therefore largely memory-bound. As the number of threads increases, the available memory bandwidth is gradually saturated, so additional threads cannot provide proportional speedup.

4.2. Use of LLM's

Chat gpt was used to properly create the code for running the strong scaling analysis. In particular, the parallelization was done by ourselves, but the code for running the strong scaling analysis and plotting the results was generated with the help of the LLM.

5. Parallel loop dependencies with `OpenMP` [15 points]

In this exercise we are asked to parallelize a loop with dependencies using `OpenMP` tasks. The code snippet of the parallelized version is shown below:

```
  #pragma omp parallel
  {
    double log_up = log(up);
    #pragma omp for firstprivate(up)
    for (n = 0; n <= N; ++n) {
      opt[n] = exp(n * log_up);
    }
    Sn = opt[N];
}
```

In table 5 is the output of the program which compares the serial and the parallel versions.

Table 5: Performance and numerical results for sequential and OpenMP implementations ($N = 2 \times 10^9$).

Implementation	Threads	Runtime [s]	Speedup	S_N	$\|\text{opt}\|_2^2/N$
Sequential	1	6.153345	1.00	4.851650976×10^8	5.884629305×10^{15}
OpenMP	8	2.890764	2.13	4.851650879×10^8	5.884629187×10^{15}

Our first approach used the function `pow` to compute the values of `opt[n]`. However, this function is not optimal and can be inefficient. By using the property that $up^n = \exp(n \log(up))$, we can compute the values more efficiently. This approach allows us to compute the powers in a more efficient way, especially for large values of n. The performance comparison in table 5 shows that the OpenMP implementation achieves a significant speedup compared to the sequential version, demonstrating the effectiveness of parallelization in this case.

6. Quicksort using OpenMP tasks [20 points]

6.1. Parallelization

Here we are asked to parallelize the quicksort algorithm using OpenMP tasks. The code snippet of the parallelized version of the quicksort algorithm is shown below:

```
void quicksort_omp(double *data, int length, int cutoff) {
  if (length <= 1) return;

  double pivot = data[0];
  double temp;
  int left = 1;
  int right = length - 1;

  do {
    while (left < (length - 1) && data[left] <= pivot) left++;
    while (right > 0 && data[right] >= pivot) right--;

    if (left < right) {
      temp = data[left];
      data[left] = data[right];
      data[right] = temp;
    }
  } while (left < right);

  if (data[right] < pivot) {
    data[0] = data[right];
    data[right] = pivot;
  }

  /* Once in a final task, continue recursively in sequential mode */
  if (omp_in_final()) {
    quicksort_seq(data, right);
    quicksort_seq(&(data[left]), length - left);
    return;
  }

  #pragma omp task shared(data) firstprivate(right, cutoff) final(right <= cutoff)
  quicksort_omp(data, right, cutoff);

  #pragma omp task shared(data) firstprivate(left, length, cutoff) final((length - left)
      <= cutoff)
  quicksort_omp(&(data[left]), length - left, cutoff);

  #pragma omp taskwait
}
```

In this implementation, we use OpenMP tasks to parallelize the recursive calls of the quicksort algorithm. The function `quicksort_omp` takes an additional parameter `cutoff`, which determines when to switch from parallel to sequential execution. If the size of the subarray is less than or equal to the cutoff, the function will execute sequentially using `quicksort_seq`. Otherwise, it will create new tasks for the left and right partitions of the array. The `final` clause is used to indicate that once a task reaches a certain depth (when the size of the partition is small enough), it should execute sequentially to avoid excessive overhead from task creation. The `taskwait` directive ensures that the parent task waits for all child tasks to complete before proceeding.

6.2. Performance analysis

First of all we tried to find an optimal value for the cutoff parameter. We ran the parallelized quicksort algorithm with different cutoff values and measured the execution time for each case.The results are shown in table 6 and obtained by taking the average of 5 different runs for each cutoff value.

N	Cutoff	Threads	Avg Time [s]	Wrong
10000000	100	64	1.3922109940	0
10000000	500	64	0.3112580360	0
10000000	1000	64	0.2273432380	0
10000000	5000	64	0.2251094760	0
10000000	10000	64	0.2258029560	0
10000000	20000	64	0.2444692820	0
10000000	50000	64	0.2295661280	0
10000000	100000	64	0.2476871600	0

Table 6: Performance of the parallelized quicksort algorithm for different cutoff values.

The cutoff value that resulted in the best performance is highlighted in yellow in the table. Based on the results, we can see that a cutoff value of 5000 provides the best performance for our implementation of the parallelized quicksort algorithm. This means that when the size of the partition is less than or equal to 5000, it is more efficient to execute the sorting sequentially rather than creating new tasks for smaller partitions. The choice of cutoff can significantly impact the performance of the parallelized algorithm, as it balances the overhead of task creation with the benefits of parallel execution.

We have then performed a strong scaling analysis by running the parallelized quicksort algorithm with the optimal cutoff value and varying the number of threads and for different values of N. The results are shown in figure 6. The data for these plots were obtained by taking the average of 5 different runs for each thread count to ensure reliable measurements.

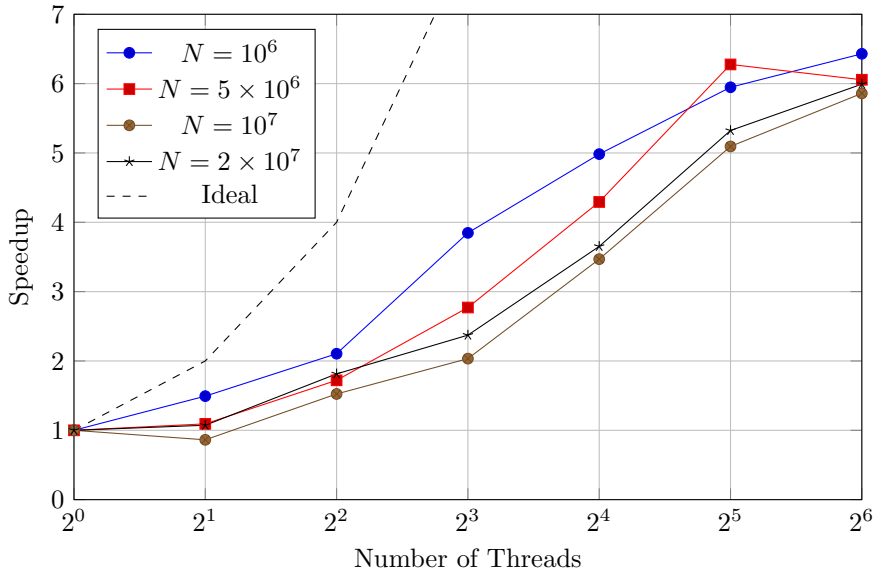


Figure 6: Strong scaling speedup for different problem sizes.